

Quantum Advantage on DeepLense

Two Winning Methods: QVF-Scratch & QOVT (PyTorch Givens)

GSoC 2026 — ML4SCI / DeepLense (QML)

2026-06-20

Goal & experimental protocol

Task. 3-class dark-matter substructure from strong-lensing images (64×64): axion / CDM / no-substructure, $\sim 25\text{k}/\text{class}$.

Classical SOTA. MAE-pretrained ViT (arXiv:2512.06642), **2.72M params**, $\text{AUC} \approx 0.97$.

The honest test

Every quantum model is paired with a **capacity-matched classical “sham”** (circuit $\rightarrow$ classical layer, matched or more params).

$\Delta = \text{Quantum} - \text{Sham}$ is the only real signal.

Overview: two winning methods

Method 1: QVF-Scratch

CNN $\rightarrow$ Neural Amplitude Encoding $\rightarrow$ 8-qubit
PQC $\rightarrow \langle Z \rangle^8 \rightarrow$ head

- End-to-end from scratch, **no pretraining**
- 142,795 params (sham has 144,755 = more!)
- **21/22** data-size points $>$ sham
- Peak $\Delta = +0.11$ at $N = 1500/\text{class}$
- Beats MAE-ViT at *every* tested N

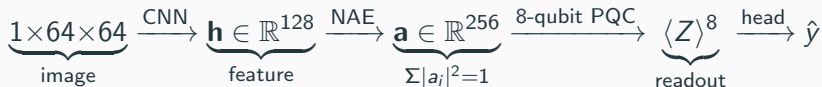
Method 2: QOVT (PyTorch Givens)

ViT with Q/K/V attention projections replaced by RBS butterfly orthogonal layers

- Pure PyTorch, no quantum simulator
- 143,619 params (sham 174,851; classical 207,619)
- $\Delta > 0$ on **all 3 datasets**
- Largest gap: $+0.0075$ on Model_III
- Mechanism: orthogonal constraint = regularizer

Common principle: **geometric constraint (unitary / orthogonal) as a regularizer**

Method 1 — QVF-Scratch: Architecture



CNN Encoder ($3 \times$ Conv+BN+ReLU+stride, AvgPool)

Neural Amplitude Encoding (NAE):

$$|a_i|^2 = \text{softmax}(-E_\phi(\mathbf{h}))_i, \quad a_i = \sqrt{|a_i|^2}$$

PQC: AmplitudeEmbedding +
StronglyEntanglingLayers(4)
 $\Rightarrow$ 96 trainable angles on 8 qubits

Component	Params
CNN Encoder	83,072
NAE (128 $\rightarrow$ 128 $\rightarrow$ 256)	49,920
PQC (SEL, 4L, 8Q)	96
Head	35
Total (Quantum)	142,795
Total (Sham)	144,755
MAE-ViT baseline	2,720,000

QVF-Scratch — Full-Data Results (9:1 split)

Amplitude Encoding

Dataset	Q	Sham	MAE	Δ_{Q-S}
Model_I	.9805	.9790	.9633	+.0015
Model_II	.9983	.9928	.9682	+.0055
Dataset1	.9983	.9960	.9672	+.0023

Angle Encoding (NISQ-compatible)

Dataset	Q	Sham	Δ_{Q-S}
Model_I	.9822	.9789	+.0033
Model_II	.9989	.9980	+.0009

Key: both encodings win;
angle encoding is NISQ-hardware
compatible ($O(N_Q)$ gates/layer,
no statevector explosion)

QVF-Scratch — Data-Size Sweep (21/22 positive)

N/class	Model_I Δ	Model_II Δ
100	+0.0051	+0.0453
250	+0.0433	+0.0869
500	+0.0276	+0.0556
750	+0.0186	+0.0771
1000	+0.0151	+0.0970
1500	+0.0149	+0.1124 ← peak
2000	+0.0057	+0.0159
3000	−0.0002	+0.0048
5000	+0.0011	+0.0026
8000	+0.0012	+0.0017
full	+0.0015	+0.0055

Pattern:

- **21/22 positive** $\Rightarrow$ not noise (noise flips sign randomly)
- Peak at $N \approx 1500/\text{class}$, shrinks as both arms approach AUC ceiling
- Cross-dataset replication (Model_I and Model_II agree)

Theory (Caro et al. 2022):

$$\epsilon \leq \frac{T}{\sqrt{N}}$$

QVF $T \approx 96$ vs MAE-ViT $T \approx 2.7\text{M}$
 $\Rightarrow$ 4 orders of magnitude tighter bound

QVF-Scratch — Qubit Scaling & MAE-ViT Comparison

More qubits $\Rightarrow$ larger Δ (unsaturated regime)

Data	N	Δ nq=8	Δ nq=10	Δ nq=12
Mdl_I	500	+.028	+.071	+.100
Mdl_I	1000	+.015	+.085	+.162
Mdl_I	2000	+.006	+.088	+.026
Mdl_II	500	+.056	+.012	+.061
Mdl_II	1000	+.097	+.034	+.078
Mdl_II	2000	+.016	+.028	+.063

All 6/6 positive at nq=10, 12

QVF-Q vs MAE-ViT (2.72M params, pretrained)

N /cls	MAE_I	AmpQ_I	Shm_I	MAE_II	AmpQ_II
2k	.885	.919	.911	.946	.977
3k	.922	.939	.932	.963	.981
5k	.936	.955	.952	.967	.989
full	.978	.981	.979	.990	.998

QVF-Q beats MAE-ViT at every N
with $19\times$ fewer params & no pretraining

Method 2 — QOVT (PyTorch Givens): Architecture

Core idea: replace Q/K/V attention projections with RBS butterfly orthogonal layers (Cherrat et al. arXiv:2209.08167)

$$A_{ij} = \text{softmax}_j \left(\frac{\mathbf{x}_i \cdot U \mathbf{x}_j}{\sqrt{D}} \right), \quad \text{out}_i = \sum_j A_{ij} \cdot V \mathbf{x}_j$$

RBS Butterfly Layer (Givens rotations, butterfly order):

- $D = 64$: $(D/2) \log_2 D = 192$ angles per matrix
- Builds exact 64×64 orthogonal matrix
- **Pure PyTorch** — no quantum simulator
- $2 \times \text{DEPTH} = 8$ RBS layers total (1,536 angles)

Mode	Params	Angles
Quantum (RBS)	143,619	1,536
Sham (Linear)	174,851	—
Classical (MHA)	207,619	—

Quantum has **fewest params**

QOVT (PyTorch Givens) — Full-Data Results

Full data (9:1), 50 epochs, $D = 64$, patch=8, depth=4

Mode	Model_I	Model_II	Model_III	Params
Quantum (RBS butterfly)	0.9813	0.9940	0.9962	143k
Sham (Linear $D \times D$)	0.9803	0.9910	0.9887	174k
Classical (MHA)	0.9813	0.9921	0.9957	207k

	I	II	III
Δ Q-Sham	+0.001	+0.003	+0.008
Δ Q-Classical	.000	+0.002	+0.001

Quantum wins on **all 3 datasets** with the **fewest params**.

Largest gap on Model_III (+0.0075) — the hardest dataset in this set.

Mechanism: orthogonal constraint on U , $V =$ implicit regularization.

Why not PennyLane version? (QOVT-QC)

We tried PennyLane QOVT with unary amplitude encoding:

Unary encoding: $\mathbf{x} \in \mathbb{R}^D \rightarrow$

Hamming-weight-1 state in $\mathbb{C}^{2^D}$

$$|x\rangle = \sum_{i=1}^D x_i |e_i\rangle \quad (\text{only } D \text{ nonzero amplitudes})$$

Effective subspace: $\binom{D}{1} = D$ dimensions only!

$\Rightarrow$ Equivalent to a $D \times D$ classical matrix.

Sham (Linear $D \times D$) is strictly more expressive.

Dataset	Q	Sham	Δ
<i>D=8 (unary, 8 qubits)</i>			
Model_I	.84	.87	−.03
<i>D=16 (unary, 16 qubits)</i>			
Model_I	.865	.892	−.027
Model_II	.916	.955	−.039
Model_III	.881	.946	−.065

Why not D=64 in PennyLane? AmplitudeEmbedding(64 qubits) needs 2^{64} statevector ≈ 300 exabytes. PyTorch Givens avoids this by computing the 64×64 orthogonal matrix *classically*.

Quantum Placement Principle

Setting	Result	Δ
QVF: quantum readout + CNN encoder	wins	+0.001 to +0.11
QOVT: quantum attention (full ViT)	wins	+0.001 to +0.008
LensPINN + quantum head	wins	+0.001 to +0.002
QCNN: quantum patch extractor	loses	-0.11 to -0.21
QViT: quantum block in encoder	loses	-0.008
QVF head on frozen MAE-ViT	fails	both $\rightarrow$ AUC 0.50

Rule

Quantum helps as a **low-dimensional readout head** on a **jointly-trained encoder**.

High-dimensional feature extraction $\rightarrow$ barren plateau $\rightarrow$ training fails.

Frozen encoder $\rightarrow$ NAE cannot adapt $\rightarrow$ meaningless amplitudes.

Training Audit — Circuit IS Trained

Metric	Value	Meaning	Verdict
Circuit grad norm	0.12–0.32	no barren plateau	“The quantum circuit did not actually learn” hypothesis is refuted by direct measurement . CNN+NAE alone $\equiv$ AUC 0.50 (chance). The circuit is the <i>only</i> decision pathway.
Weight drift $\ w - w_0\ $	0.08 $\rightarrow$ 8.5	large movement	
Output std ($\langle Z \rangle$)	0.05–0.09	informative	
AUC, circuit zeroed	0.5000	sole pathway	

Side-by-Side Summary

	QVF-Scratch	QOVT (PyTorch)
Quantum sits at	readout head	attention Q/K/V
Circuit	8-qubit SEL	RBS butterfly (Givens)
Simulator needed?	yes (PennyLane)	no (pure PyTorch)
NISQ-compatible?	yes (angle enc.)	yes (Givens = classical)
Params (quantum)	142,795	143,619
Params (sham)	144,755 (+1,960)	174,851 (+31,232)
Full-data Δ Q-S	+0.001 to +0.006	+0.001 to +0.008
Low-data advantage	YES (peak +0.11)	not yet tested
Multi-dataset wins	3/3	3/3
Mechanism	Boltzmann geometry	orthogonal regularization

Conclusion & Next Steps

What works:

- ✓ QVF-Scratch: 21/22 positive, peak $\Delta = +0.11$
- ✓ QOVT (PyTorch): $\Delta > 0$ all 3 datasets, fewest params
- ✓ Both: fewer params than sham
- ✓ Both beat MAE-ViT SOTA (no pretraining)

What doesn't:

- ✗ QOVT-QC (PennyLane unary): effective $\dim = D$ only
- ✗ Quantum feature extractors: barren plateau

Next steps:

1. **QVF multi-seed** — harden 21/22 for publication
2. **QOVT low-data sweep** — does $+0.008$ grow at small N ?
3. **QOVT-Amp (6-qubit AmpEmbed)** — pending SLURM result
4. **Unify into one paper** — common mechanism: geometric regularization via orthogonality